MODULE 04 | PERMISSIONS

Permissions

Control who can read, write and execute every file, the foundation of Linux security.

Mission	Topic	Commands / Concepts
1	Read the Permissions: ls -l, id and groups	ls -l, id, groups, whoami
2	Change Permissions: chmod	chmod, symbolic, octal, -R
3	Ownership: chown and chgrp	chown, chgrp, sudo, -R
4	Defaults and Special Bits: umask	umask, setuid, setgid, sticky bit
Boss	Lock It Down	Everything in this module

How to use this guide: read each mission, type every example yourself in the Tux Tutor terminal, attempt the practice tasks, then check the answers at the end. Use a throwaway practice folder so mistakes cost nothing.

Module Overview

Why permissions matter

Linux is a multi-user system. Permissions stop one user from reading another's files, stop programs from changing system files, and decide which scripts may run.

Three actors, three rights

Actor	Letter	Who
User (owner)	u	The person who owns the file
Group	g	Members of the file's group
Others	o	Everyone else
All	a	u + g + o together

Right	Letter	On a file	On a directory
Read	`r`	View contents	List names inside (`ls`)
Write	`w`	Modify contents	Create, delete, rename files inside
Execute	`x`	Run as a program	Enter it (`cd`) and access items

Directories are different
A folder needs `x` to be entered. Without `x`, you cannot `cd` into it or reach files inside, even if you have `r`.

Setup

```
$ mkdir -p ~/perm && cd ~/perm
$ touch file.txt script.sh && mkdir shared
```

MISSION 1 | +50 XP

Read the Permissions: ls -l, id and groups

Goal: Decode the permission string and know who you are on the system.

```
$ ls -l
-rw-r--r-- 1 haani dev 0 Nov 14 10:00 file.txt
drwxr-xr-x 2 haani dev 4096 Nov 14 10:00 shared

- rw- r-- r--   haani   dev
| |   |   |     |       |
| |   |   |     |       group
| |   |   |     owner
| |   |   others (read only)
| |   group (read only)
| user (read + write)
file type: - file, d directory, l symlink
```

Numeric (octal) values

Each right has a value: **r = 4, w = 2, x = 1**. Add them for each actor.

Symbolic	Math	Octal
rwx	4+2+1	7
rw-	4+2+0	6
r-x	4+0+1	5
r--	4+0+0	4
---	0	0

```
rw-r--r--  =  6 4 4  =  644
rwxr-xr-x  =  7 5 5  =  755
rw-------  =  6 0 0  =  600
```

Who am I?

```
$ whoami
haani
$ id
uid=1000(haani) gid=1000(haani) groups=1000(haani),27(sudo),1001(dev)
$ groups
haani sudo dev
$ ls -l /etc/shadow /etc/passwd     # compare sensitive vs public files
```

Common permission sets

Octal	Typical use
644	Normal files (owner edits, all read)
600	Private files such as SSH keys

Octal	Typical use
`755`	Programs, scripts and directories
`700`	Private directories
`775`	Shared group folders
`777`	Everyone can do everything. Avoid!

Practice

- Run `ls -l /etc/passwd /etc/shadow` and explain why they differ.
- Convert `rwxr-x---` to octal.
- Which groups are you in? Can you use sudo?

MISSION 2 | +50 XP

Change Permissions: chmod

Goal: Grant and remove rights with both symbolic and numeric modes.

Symbolic mode

Format: `chmod WHO OPERATOR RIGHTS file`, where operator is + add, - remove, = set exactly.

```
$ chmod u+x script.sh          # owner can execute
$ chmod g-w file.txt           # remove group write
$ chmod o-rwx file.txt         # others get nothing
$ chmod a+r file.txt           # everyone can read
$ chmod u=rw,g=r,o= file.txt   # set exactly: rw-r-----
$ chmod +x script.sh           # add x for all (affected by umask)
```

Numeric mode

```
$ chmod 644 file.txt          # rw-r--r--
$ chmod 755 script.sh          # rwxr-xr-x
$ chmod 600 secret.txt         # rw-------
$ chmod 700 shared             # rwx------
$ chmod -R 755 project/        # recursive for a whole tree
```

Real example: run a script

```
$ echo 'echo Hello from script' > script.sh
$ ./script.sh
bash: ./script.sh: Permission denied
$ chmod u+x script.sh
$ ./script.sh
Hello from script
```

Warning
Avoid `chmod -R 777`. It makes every file writable by everyone, which is a serious security hole. If something 'only works with 777', find the real owner or group problem instead.

Tip
A handy recursive trick: directories need x, files usually do not. Use `find . -type d -exec chmod 755 {} +` and `find . -type f -exec chmod 644 {} +`.

Practice

- Make `secret.txt` readable and writable only by you, using both a symbolic and a numeric command.
- Make `script.sh` executable and run it.
- Remove all rights from others on the folder `shared` and confirm with `ls -ld shared`.

MISSION 3 | +50 XP

Ownership: chown and chgrp

Goal: Change which user and group own a file.

chown - change owner

```
$ sudo chown bob file.txt              # owner -> bob
$ sudo chown bob:dev file.txt          # owner bob, group dev
$ sudo chown :dev file.txt             # group only
$ sudo chown -R bob:dev project/       # whole tree
$ sudo chown --reference=a.txt b.txt   # copy ownership from another file
```

chgrp - change group

```
$ chgrp dev file.txt                   # allowed if you own it AND belong to dev
$ sudo chgrp -R www-data /var/www/site
```

Who can change what?
Only **root** can change a file's owner. The owner can change the group, but only to a group the owner belongs to.

Shared folder recipe

Goal: everyone in group `dev` can read and write inside `/srv/project`.

```
$ sudo groupadd dev                    # create group
$ sudo usermod -aG dev haani           # add user (log out and in to apply)
$ sudo mkdir -p /srv/project
$ sudo chown root:dev /srv/project
$ sudo chmod 2775 /srv/project         # setgid: new files inherit group dev
```

sudo basics

```
$ sudo command           # run one command as root
$ sudo -i                # root shell (leave with exit)
$ sudo -l                # what am I allowed to run?
```

Tip
Use `sudo` only for the exact command that needs it. Working in a root shell all day makes mistakes costly.

Practice

- Run `ls -l` on `/var/log` and note owners and groups.
- Create a file, then try changing its owner to `root` with and without `sudo`.
- Create a group `team`, add yourself, and make a folder owned by `:team`.

MISSION 4 | +50 XP

Defaults and Special Bits: umask

Goal: Understand default permissions and the three special permission bits.

umask - the default mask

New files start from a base value and the **umask** removes rights from it.

Item	Base	With umask 022	Result
New file	666	666 - 022	644 (rw-r--r--)
New directory	777	777 - 022	755 (rwxr-xr-x)
New file	666	666 - 077	600 (private)

```
$ umask             # show current mask
0022
$ umask 077         # new files private to you (this session only)
$ touch private.txt && ls -l private.txt
-rw------- 1 haani haani 0 Nov 14 private.txt
$ umask -S          # symbolic form: u=rwx,g=,o=
```

Tip
Make a umask permanent by adding `umask 027` to `~/.bashrc`.

Special permission bits

Bit	Octal	Effect	Example
setuid	4000	Program runs as its **owner**	`/usr/bin/passwd` (shown `rws`)
setgid	2000	Runs as group / new files inherit directory group	Shared folders (`rws` on group)
sticky	1000	In a directory, only the file's owner can delete it	`/tmp` (shown `rwt`)

```
$ ls -ld /tmp
drwxrwxrwt 15 root root 4096 Nov 14 /tmp
$ chmod +t shared             # sticky bit
$ chmod g+s shared            # setgid
$ chmod 1777 shared           # sticky + rwxrwxrwx
$ find / -perm -4000 -type f 2>/dev/null   # audit setuid programs
```

Warning
setuid files run with their owner's power. Unexpected setuid root programs can indicate a security problem.

Practice

- Set `umask 077`, create a file and a directory, and read their permissions.
- Why does `/tmp` show a `t` at the end?
- List the setuid programs on your system and identify `passwd`.

BOSS BATTLE | +150 XP

Boss: Lock It Down

Goal: combine everything from this module without hints.

Scenario

You manage a small team folder. Alice and Bob (group `team`) must collaborate, others must be blocked, and a private notes file must stay private.

Tasks

1. Create `~/vault` containing `public.txt`, `team.txt`, `private.txt` and `run.sh`.
2. Make `private.txt` readable and writable by you only.
3. Make `public.txt` readable by everyone but writable only by you.
4. Make `run.sh` executable by you and your group, but not by others.
5. Make the `vault` directory accessible only by you (`rwx------`).
6. Set `umask 027` and create `new.txt`; confirm its permissions.
7. Convert the permissions `rw-r-x---` and `rwxrwxrwx` to octal.
8. Explain why `chmod 777` would be a poor fix for a permission denied error.

Sample solution

```
mkdir ~/vault && cd ~/vault
touch public.txt team.txt private.txt run.sh
chmod 600 private.txt
chmod 644 public.txt
chmod 750 run.sh
chmod 700 ~/vault
umask 027 && touch new.txt && ls -l new.txt   # rw-r-----
# rw-r-x--- = 650 ; rwxrwxrwx = 777
ls -l
```

Check Your Understanding

Q1. What do r, w, x equal numerically?

Q2. What does `chmod 640` mean?

Q3. What does x mean on a directory?

Q4. Who can change a file's owner?

Q5. What does `chmod u+x file` do?

Q6. Default file permissions with umask 022?

Q7. What is the sticky bit used for?

Q8. What does setuid do?

Q9. Why avoid `chmod 777`?

Q10. Command to add a user to a group?

Answers

#	Answer
1	4, 2, 1.
2	rw-r----- (owner rw, group r, others none).
3	Permission to enter it and access items inside.
4	Only root (via sudo).
5	Gives the owner execute permission.
6	644.
7	Only owners may delete their files in a shared directory like /tmp.
8	Runs a program with its owner's privileges.
9	It lets everyone read, write and run the file, which is a security risk.
10	sudo usermod -aG group user

Quick Reference Cheat Sheet

Command	Purpose	Example
ls -l	View permissions	ls -ld folder
chmod u+x	Add execute for owner	chmod u+x run.sh
chmod 644	Numeric set	chmod 644 f
chmod -R	Apply recursively	chmod -R 755 dir
chown u:g	Change owner and group	sudo chown bob:dev f
chgrp	Change group	chgrp dev f
umask	Default mask	umask 027

Command	Purpose	Example
id / groups	Who am I	id
chmod +t / g+s	Sticky / setgid	chmod g+s dir
sudo	Run as root	sudo cmd
find -perm	Audit permissions	find . -perm 777

Up next: Module 05
Processes. Use ps, top, kill, jobs, nice and nohup to watch and control running programs.